

ClaraScript

a delphi experiment

ClaraScript is a plain-text notation for recipes, in order to be rendered as a CFE-style instruction table. A big thanks to Michael Chu for inventing the CFE format.

Document Model

A ClaraScript recipe carries a **head section** and a **body section**. The body section is a sequence of **chains**, which contain a run of **steps**, each executing sequentially from the step before it.

A step has to take at least one **ingredient** as an input, and may name its output so a later step can **reference** it.

The result of a parsed ClaraScript recipe is a tree. Each step has exactly one downstream consumer which makes the table easily drawn.

To parse and make a ClaraScript-style CFE diagram for any given recipe, use this reference to draw it or use the Recipe Table experiment available on the web on **delphi.tools**.

Lines

Each line takes one of eight valid forms. Leading and trailing whitespace is stripped and indentation carries no meaning. The following formats are valid:

An **empty line** ends the current chain.

A line prefixed with **title:**, **serves:** or **units:** is considered part of the document metadata.

A line prefixed with a **greater than sign (>)** denotes a note, which is rendered above the diagram.

A line **without a prefix** is considered a step. A step is required for the following lines, which all need to follow a step.

A **hyphen (-)** denotes which **ingredient** is used for the step.

The **letter x** as a prefix denotes discarding part of the step's output.

The **equals sign (=)** names the output of this particular step.

ClaraScript

a delphi experiment

An **@ symbol** denotes an ingredient being a **reference** to a previously named output.

A **pipe character (|)** separates a label from its **details**, provided the line has details. It never means anything else.

An attachment line that does not follow a step or an attachment is considered an error. The discard instruction (x) requires a space after the x, so "xanthan gum" isn't parsed as a discard instruction.

serves is set to 2, then asking for 3 at compile time gives a factor of 1,5. Without a serves key, it is assumed that the recipe serves 1.

The units key takes either "**metric**" or "**imperial**". It is optional and records which mode of measurement the recipe was written in. Without it, the compiler will refuse to convert units even if asked.

Keys are case insensitive and whitespace before or after the colon character is ignored.

Frontmatter

Metadata is only recognised before the first step. After the first step has been defined, colons are ignored by the interpreter and become text.

title: Aglio e olio

serves: 2

units: metric

The title field takes the rest of the line verbatim, and names the document. The **serves** field takes a positive number. In addition to its communicative value, it also sets the denominator for **scaling**. If

Notes

> Salt a large pot of water

Notes are numbered in document order and are listed above the diagram. A note may also mark a position within the recipe flow and can attach to the next step written after it, which makes the compiler render it on the leading edge of that step. Notes written above the first step are not shown as badges but rather as plaintext numbered instructions.

ClaraScript

a delphi experiment

Recipe Body

Steps and chains

As established before, each unprefixed line counts as a step. Its first field is always the steps instruction, and any further fields are details.

fry | 2 min

boil | 9 min | covered

A blank line will start a new chain on the next line. Inside a chain, each step consumes the step before it, and that predecessor becomes its input.

boil | 9 min

- spaghetti: 200g

drain

toss | 1 min

serve

The last chain in the document is the **finished product**, also called the root. Every other chain must be reached by **reference**, or is reported as unused by the compiler and does not appear in the diagram.

Ingredients

An ingredient is written **name first, quantity second**, split on the first colon.

- **spaghetti: 200 g**
- **garlic: 2 cloves**
- **water: salted**
- **parsley**

Everything left of the colon is considered the name. Everything to the right of the colon is considered as the quantity, which is parsed for scaling when it can be and considered text when it can't.

Only the first colon will split a line. Therefore, "**- stock: 1 l | ratio 2:1**" will return the name "**stock**", the quantity "**1 l**" and the detail "**ratio 2:1**". The current implementation of the renderer displays the ingredient as it's quantity, meaning "**- spaghetti: 200g**" will be rendered as "**200 g spaghetti**".

ClaraScript

a delphi experiment

Prep

Using the slash character on an ingredient line, surrounded by spaces, will add a one-input step (a "**monostep**") applied to that ingredient alone.

- **garlic: 2 cloves / slice**
- **parsley: 1 handful / chop | fine**

The separator must be surrounded by spaces to take effect, as to not affect strings like "**1/2**" or "**salt/pepper**".

Named outputs

The equals sign **names** the output of the step above it. This is essential to chain multiple ingredient production lines together. Using an **@ symbol** introduces it to another chain.

fry | 2 min

- **olive oil: 2 Tbsp**
 - **tomatoes: 2 cans**
- = sauce**

combine

- **spaghetti: 200g**
- @ sauce**

Notably, names resolve backwards only. If a name is called on in a later step, it must have been defined earlier in the document. This keeps the tree acyclic and avoids cycle checking in the compiler.

A reference may resolve in one of two ways. If it is in a different chain to the target operation, it will **inline**, and will have its subtree drawn in place. Otherwise, it will become a **linked row**. This covers a reference to a step in the same chain, which is usually a second use of an already inlined chain, which may be useful for byproducts such as pasta water or fonds.

A reference respects the detail parameter, which is the usual place for the portion being used.

loosen

@ pasta water | half cup

A named output still flows to the next step in its chain. Naming an output does not change the order of operations.

ClaraScript

a delphi experiment

Discarding

drain

x half the water

A discard is a note on the step's output, rendered struck through by default. It is not referencable and takes no detail parameters.

Quantities

ClaraScript accepts the following number formats: **Integer** (200), **decimals** as point or comma (1,5 and 1.5), **fractions** (1/2), **whole plus fractions** (1 1/2), as well as **ranges** (2-3). Quantities joined by a plus sign are scaled by term, meaning "1 cup + 1 Tbsp" will be scaled individually.

Units

ClaraScript supports the following units:

Volume: ml, dl, l, tsp, Tbsp, cup, fl oz, pint, quart, gallon

Mass: g, kg, oz, lb

Length: mm, cm, in

Long forms and plurals are generally accepted. Case is ignored except for tablespoons and teaspoons, Tbsp has to be capitalised.

Unknown units are treated as **counts**. Counts scale to halves and not to decimals. Text the parser does not recognise as a unit are left as written. "**a pinch**" will scale to "**a pinch**" no matter how many servings are desired.

Scaling is a **compiler operation**. The document will always represent the intent with which it was written. If your compiler supports unit conversion, refer to its notes on how it handles mixed units.

Rounding follows the unit's convention. Grams round to 1 below 100 and to 5 above, kilograms to 0.05, cups to 1/8, tablespoons to 1/2 and teaspoons to 1/8. A positive amount will never round to zero.

If ingredient lists are rendered, they are converted to a base unit, added, scaled, and presented once in the ingredient list.

ClaraScript

a delphi experiment

Example recipes

title: Potato Soup

serves: 6

units: imperial

boil | 10 min

- onions: 4 / peel and chop
- dripping fat: 2 oz
- water: 3 quarts

stir | 25 min | with a wooden spoon
| until the potatoes are done to a
pulp

- potatoes: 4 lb / peel and slice
- pepper
- salt

title: Pancake

serves: 1

units: metric

heat

- butter: a bit
- = pan

mix

- wheat flour: 200 g
- milk: 200 ml
- eggs: 2
- sugar: 2 tsp

- baking soda: 1 tsp

- salt: a little bit

rest | ~15 min

bake | a ladle at a time | lower

medium heat | until brown on both
sides

@ pan

> Serve with syrup, or cinnamon
and sugar

title: Miso Soup

units: metric

boil

- water: about half

add

- dashi stock

boil

- tofu: optional / cut into bite-
sized pieces

- seaweed: optional / cut into bite-
sized pieces

melt

- miso paste

boil

serve

- green onion: optional / slice